IN432 • ADVANCED WEB TECHNOLOGIES

JavaScript

Hello World → Two Independent Applications

Build JavaScript on a separate frontend server; send JSON CRUD requests to the existing Session 5 Flask API.



NO REACT • SEPARATE FRONTEND :5500 • FLASK BACKEND :5000

What you will be able to do

Build a standalone JavaScript frontend that communicates with an independent Flask API.

OUTCOMES

Write JavaScript

Run expressions, variables, functions and loops in the browser.

Change a webpage

Read inputs, react to user events, and render task items.

Talk to an API

Use fetch and JSON with GET, POST, PATCH and DELETE on another origin.

The browser and Flask run separately. They need an HTTP API contract, not shared source files.

The learning path

One task manager grows at each stage.

1

Hello World

Display a message

2

JavaScript

Compute with data

3

DOM

React to clicks

4

HTTP + JSON

Read server data

5

CRUD

Create, update, delete

Concept → small code example → student activity → working checkpoint.

The final goal

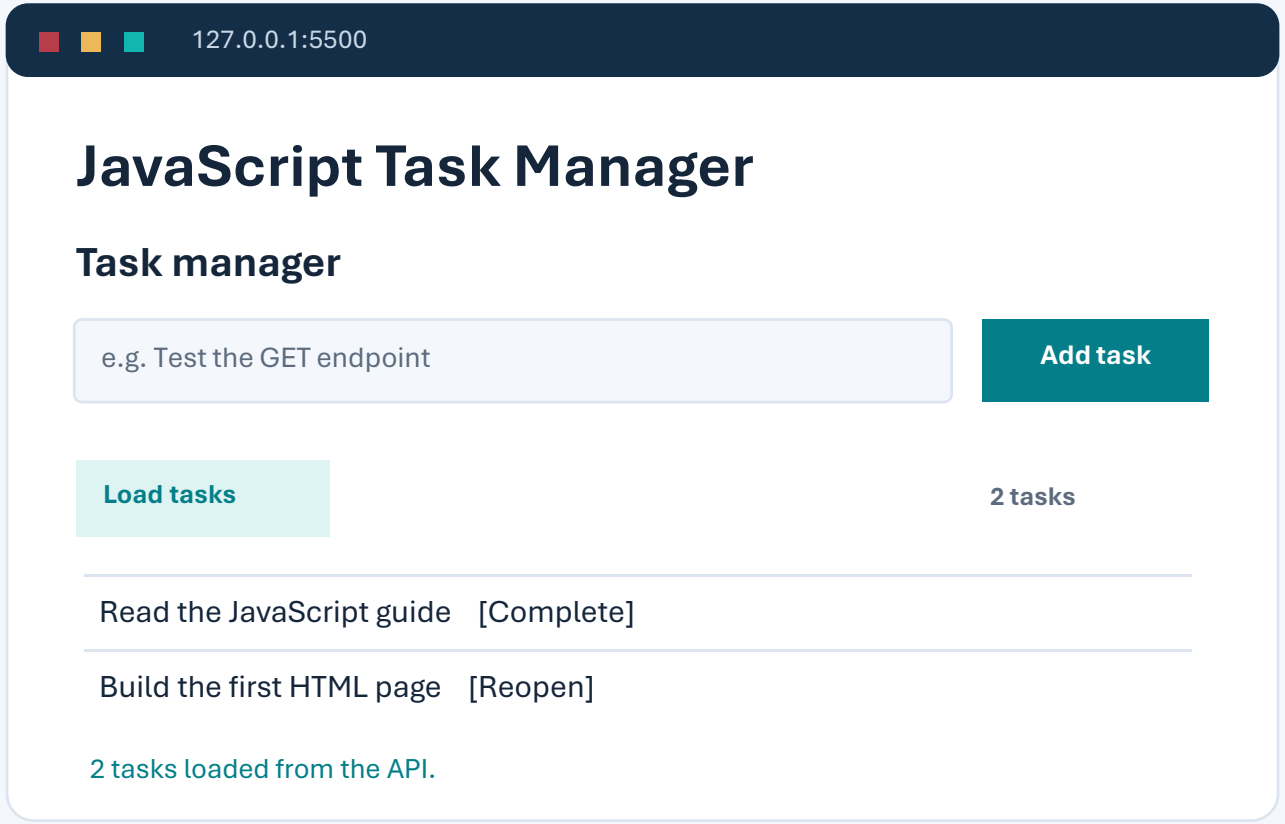
Two separately hosted apps communicate through a documented JSON API.

CHECKPOINT

SOURCE TO COMPARE

Reference: solution/app.js

- Read tasks from Flask GET /api/tasks
- Create tasks in SQLite using JSON POST
- Update with PATCH and remove with DELETE



What we assume—and what we do not

Students are already comfortable with HTML and CSS.

YOU KNOW	YOU WILL LEARN
HTML tags and attributes	JavaScript variables, objects and arrays
CSS selectors and styling	Selecting and changing DOM elements
Forms and buttons in HTML	Event listeners and validation
How to open a webpage	Asynchronous requests with fetch

No React or Node.js required at this stage. You already know the Flask backend from Session 5.

Your tools

Two folders and two terminals: no frontend files inside Flask.

SETUP

Browser + DevTools

The Console shows values and errors. The Network panel shows HTTP requests.

Code editor

Edit frontend/app.js; HTML and CSS are provided in the standalone folder.

Independent Flask API

Terminal A (backend/): python -m flask --app task_api run --debug --port 5000

Terminal B (frontend/): python -m http.server 5500 --bind 127.0.0.1; visit port 5500.

01

Start with Hello World

One statement. One observable result.

Goal: run JavaScript and locate its output.

Three languages, three jobs

A browser page can combine HTML, CSS and JavaScript.

LINK

HTML → structure

Contains the h1, button, form and list elements.

CSS → appearance

Controls colors, spacing, sizes and layout.

JavaScript → behavior

Changes text, handles clicks and requests API data.

Today we write JavaScript, not JSX or React.

What happens when a webpage loads?

Read the path from file to visible interface.

1

Browser

Requests index.html

2

HTML

Creates page structure

3

CSS

Styles elements

4

JavaScript

Adds behavior

The browser executes client-side JavaScript. The Session 5 Flask API is a separate server-side part.

Console, source file, terminal

The browser Console runs JS; separate terminals run Flask and the static frontend server.

WORKSPACE	USED FOR	EXAMPLE
Browser Console	Try a JavaScript expression immediately	console.log(2 + 3)
frontend/app.js	Save code that runs with the webpage	const name = "Lina";
Terminal A / B	Start backend server / start frontend server	flask run --port 5000 / http.server 5500

Run commands in the appropriate folder, never in the browser Console.

Our first JavaScript statement

The browser Console is a safe place for tiny experiments.

READ THE CODE

- console is a browser object.
- log(…) displays the value.
- Quoted text is a string.

BROWSER CONSOLE

```
console.log("Hello, World!");
```

Checkpoint 1 — Hello World

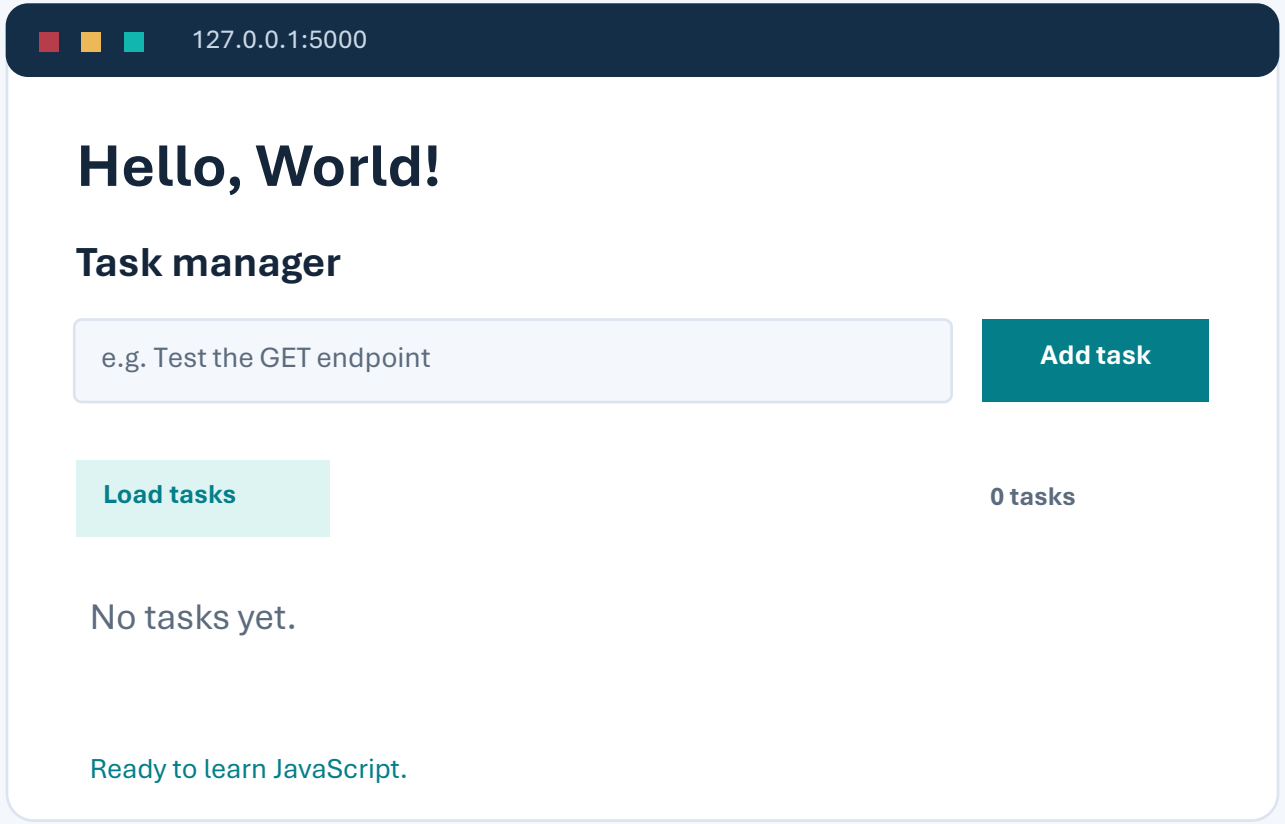
Open DevTools → Console and inspect the output.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/01_hello.js

- Expected log: Hello, World!
- The webpage still has its original heading
- No API request is needed yet



Practice 1 — Run three statements

Write and predict the output before pressing Enter.

YOUR TURN

TASK

1. Print your first name using console.log.
2. Calculate and print 7 + 5.
3. Print the string "7 + 5" without calculating it.

EXPECTED RESULT

Your name, 12, then the text 7 + 5.

HINT / CHECK

Quotes mean text; unquoted 7 + 5 is arithmetic.

Solution — First Console practice

Compare your code only after attempting the exercise.

READ THE CODE

- Names inside quotes are strings.
- The second line adds two numbers.
- The third prints literal text.

ANSWER / ONE POSSIBLE SOLUTION

```
console.log("Lina");
console.log(7 + 5);
console.log("7 + 5");
```

From Console to app.js

JavaScript code lives in frontend/app.js, loaded from the independent frontend site.

CONNECTION

HTML loads JS

A script element points to the JavaScript source file.

defer matters

The browser parses the document before running the deferred script.

Reload to retest

Edit the file, save it, then refresh the browser to run the new script.

The supplied index.html already contains <script defer src="app.js"></script>.

The script link in HTML

The frontend static server delivers HTML, CSS and JS; Flask serves only the API.

READ THE CODE

- Place this code in index.html.
- config.js runs before app.js.
- defer allows both scripts to run after HTML is parsed.

FRONTEND/INDEX.HTML

```
<head>
  <link rel="stylesheet" href="styles.css">
  <script defer src="config.js"></script>
  <script defer src="app.js"></script>
</head>
```

The first line in a real file

The browser executes app.js on port 5500; Flask never executes browser JavaScript.

READ THE CODE

- This is plain JavaScript.
- Nothing is rendered automatically.
- Look in the Console—not the page.

APP.JS

```
console.log("Hello, World!");
```

Practice 2 — Make your own message

Use the project rather than the one-line Console.

YOUR TURN

TASK

1. Start the server with : python -m flask --app task_api run --debug.
2. Open http://127.0.0.1:5000.
3. Change the app.js message to "Learning JavaScript".
4. Refresh and find the new log.

EXPECTED RESULT

The Console displays Learning JavaScript.

HINT / CHECK

If nothing changes, check the path and save the correct app.js.

The first debugging habit

When output is missing, check these four things.

1

File

Did you edit app.js?

2

Save

Did the editor save?

3

Reload

Did the browser refresh?

4

Console

Any red error message?

A red Console error is evidence: read the message and its line number.

02

JavaScript essentials

Values, variables, decisions and reusable behavior.

Goal: understand each statement before writing application code.

What is a variable?

A variable is a named binding to a value.

CONCEPT

const

A binding you will not reassign. Prefer this by default.

let

A binding that may be assigned a different value.

Names

Names are case-sensitive: taskTitle and tasktitle are different.

const protects the binding, not every property inside an object.

Variables: code and output

Read each line before running it.

READ THE CODE

- const stores a name.
- let allows reassignment.
- = assigns; it is not an equality test.

BROWSER CONSOLE

```
const appName = "Task Lab";
let completed = 1;
completed = completed + 1;
console.log(appName);
console.log(completed); // 2
```

const vs let

Choose based on reassignment, not on the value type.

EXAMPLE	VALID?	WHY?
const title = "Plan";	Yes	No reassignment is requested
title = "Build";	No	Cannot reassign a const binding
let count = 1; count++;	Yes	let permits reassignment
const tasks = []; tasks.push(1);	Yes	The array object can still change

A const object is not automatically immutable.

Practice 3 — Task counter

No webpage changes yet—inspect the Console.

YOUR TURN

TASK

1. Declare const total = 7.
2. Declare let completed = 2.
3. Increase completed by 1.
4. Calculate remaining and print it.

EXPECTED RESULT

The remaining number is 4.

HINT / CHECK

Use total - completed after updating completed.

Solution — Task counter

Compare your code only after attempting the exercise.

READ THE CODE

- Each statement runs top to bottom.
- The final value of completed is 3.

ANSWER / ONE POSSIBLE SOLUTION

```
const total = 7;
let completed = 2;
completed = completed + 1;
const remaining = total - completed;
console.log(remaining); // 4
```

The values you will use

JavaScript determines types from their values.

TYPES

String

"Read the guide" — text inside quotes.

Number

3 or 5.5 — arithmetic values.

Boolean

true or false — useful for task completion.

null and undefined

null is an explicit empty value; undefined often means no assigned value.

A variable can hold a different type later; we still write predictable programs.

Strings and numbers

A plus sign has different meanings with strings.

READ THE CODE

- Numbers can be added.
- Strings can be joined.
- HTML input values are strings.

BROWSER CONSOLE

```
console.log(2 + 3);      // 5
console.log("2" + "3");  // "23"
console.log("Task: " + "Read");

const title = "Read";
console.log("Task: " + title);
```

Template literals make strings readable

A backtick string can insert a variable.

CONCEPT

Quotation marks

"Hi" and 'Hi' produce strings.

Backticks

`Hi` creates a template literal.

Interpolation

${variable} inserts its value into a template literal.

The ${...} syntax works inside backticks, not ordinary quotation marks.

A task summary using interpolation

The output changes with the variable.

READ THE CODE

- Backticks surround the whole string.
- ${...} evaluates an expression.

BROWSER CONSOLE

```
const title = "Plan the UI";
const completed = false;
console.log(`Task: ${title}`);
console.log(`Completed: ${completed}`);
```

Operators you need for task logic

We use strict comparisons to avoid surprising coercions.

OPERATOR	MEANING	EXAMPLE
+ - * /	Arithmetic	5 - 2 gives 3
=== !==	Strict equality / inequality	done === false
> < >= <=	Number comparisons	title.length > 0
&& || !	AND / OR / NOT	!done
=	Assignment	done = true

Use === for equality; do not confuse it with assignment (=).

A condition chooses a path

The condition is evaluated to decide what happens.

CONCEPT

if

Runs a block when the condition is truthy.

else

Runs the alternative block.

Braces

Group the statements belonging to each branch.

In our task example, done is an actual Boolean—not the string "false".

Mark a task Active or Completed

Read the condition before the branches.

READ THE CODE

- The test is true or false.
- Exactly one branch runs.
- Change done to true and rerun.

BROWSER CONSOLE

```
const done = false;
let label;
if (done === true) {
  label = "Completed";
} else {
  label = "Active";
}
console.log(label); // Active
```

Practice 4 — Classify a score

Use the same decision pattern.

YOUR TURN

TASK

1. Declare let score = 72.
2. If score >= 50, print "Pass".
3. Otherwise print "Retry".
4. Change score to 40 and test again.

EXPECTED RESULT

First Pass, then Retry.

HINT / CHECK

Use if (score >= 50) {...} else {...}.

Solution — Classify a score

Compare your code only after attempting the exercise.

READ THE CODE

- Try score = 40 after the first run.
- A comparison creates a Boolean result.

ANSWER / ONE POSSIBLE SOLUTION

```
let score = 72;
if (score >= 50) {
  console.log("Pass");
} else {
  console.log("Retry");
}
```

A function packages reusable work

Define once; call when a result is needed.

CONCEPT

Parameter

A local name for input: done.

Argument

The actual value supplied when calling: true.

return

Sends a value back to the caller. It is different from printing.

Defining a function does not automatically call it.

Your first function

One function can label many tasks.

READ THE CODE

- done is a parameter.
- false is an argument.
- return gives the caller a value.

BROWSER CONSOLE

```
function statusLabel(done) {
  if (done === true) {
    return "Completed";
  }
  return "Active";
}
console.log(statusLabel(false));
```

return vs console.log

Both are useful, but they do different jobs.

CODE	RESULT
console.log("Active");	Displays text in the Console
return "Active";	Provides text to the function caller
const x = statusLabel(false);	Stores returned text in x
console.log(x);	Displays the saved value

Do not replace return with console.log when a caller needs to use the result.

Practice 5 — A reusable total

Write and call a function.

YOUR TURN

TASK

1. Write remainingTasks(total, completed).
2. Return the difference.
3. Call remainingTasks(8, 3).
4. Print the returned value.

EXPECTED RESULT

The Console shows 5.

HINT / CHECK

Do not place console.log in place of return.

Solution — Remaining tasks

Compare your code only after attempting the exercise.

READ THE CODE

- The return value is a number.
- The function can accept new inputs.

ANSWER / ONE POSSIBLE SOLUTION

```
function remainingTasks(total, completed) {
  return total - completed;
}
const result = remainingTasks(8, 3);
console.log(result); // 5
```

03

Data structures

From one task object to a collection of tasks.

Goal: model real application data with objects and arrays.

An object groups named values

One task has several related properties.

CONCEPT

Property

A name/value pair, such as title: "Plan".

Dot access

task.title reads the title.

Mixed types

An id may be a number, a title a string, and done a Boolean.

This is an object literal, not a Java class declaration.

Describe one task as an object

Curly braces hold properties.

READ THE CODE

- id identifies the task.
- title contains human-readable text.
- done records completion.

BROWSER CONSOLE

```
const task = {
  id: 1,
  title: "Plan the interface",
  done: false
};
console.log(task.title);
console.log(task.done);
```

Practice 6 — Read a property

Modify the task object.

YOUR TURN

TASK

1. Create a task with id 7.
2. Give it title "Study JSON" and done false.
3. Print the title and id.
4. Set task.done = true and print it.

EXPECTED RESULT

Study JSON, 7, true.

HINT / CHECK

const task does not prevent updating its properties.

Solution — Update an object property

Compare your code only after attempting the exercise.

READ THE CODE

- Read a property with dot notation.
- Assign a new property value with =.

ANSWER / ONE POSSIBLE SOLUTION

```
const task = {
  id: 7, title: "Study JSON", done: false
};
console.log(task.title, task.id);
task.done = true;
console.log(task.done); // true
```

An array stores an ordered collection

A task manager needs more than one task.

CONCEPT

Square brackets

[ ] create an array.

Indexing

tasks[0] reads the first element. Indexes start at zero.

length

tasks.length gives the element count.

Each item can be an object with its own id, title and done value.

An array of task objects

Read both the collection and one element.

READ THE CODE

- The outer [ ] creates an array.
- Each { } creates an object.
- Array positions start at 0.

BROWSER CONSOLE

```
const tasks = [
  { id: 1, title: "Plan", done: false },
  { id: 2, title: "Review", done: true }
];
console.log(tasks.length);      // 2
console.log(tasks[0].title);    // Plan
```

Three ways to visit items

Choose a loop based on the job.

SYNTAX	GOOD FOR
for (let i = 0; ...; i++)	Visit by numeric index
for (const task of tasks)	Read each task value directly
while (condition)	Repeat while a condition remains true
tasks.map(callback)	Create a new array of transformed values

We will use for...of to build DOM rows and map to transform data.

A for...of loop

Read each task without managing an index.

READ THE CODE

- tasks is the array.
- task is one object per iteration.
- The loop body runs twice.

BROWSER CONSOLE

```
for (const task of tasks) {
  console.log(task.title);
}
// Plan
// Review
```

An indexed for loop

Useful when the position matters.

READ THE CODE

- Initialize i = 0.
- Check i < tasks.length each time.
- Increment i after each iteration.

BROWSER CONSOLE

```
for (let i = 0; i < tasks.length; i++) {
  console.log(i, tasks[i].title);
}
// 0 Plan
// 1 Review
```

A while loop

Be sure the loop makes progress.

READ THE CODE

- Condition checked before each run.
- The body must advance i.
- No infinite loop is needed to learn this.

BROWSER CONSOLE

```
let i = 0;
while (i < tasks.length) {
  console.log(tasks[i].title);
  i = i + 1;
}
```

Practice 7 — Count active tasks

Combine an array, loop, condition and function.

YOUR TURN

TASK

1. Write countActive(tasks).
2. Start a counter at zero.
3. Visit each task and count done === false.
4. Return the total and print it.

EXPECTED RESULT

For Plan(false) and Review(true), return 1.

HINT / CHECK

return belongs after the loop, not inside it.

Solution — Count active tasks

Compare your code only after attempting the exercise.

READ THE CODE

- count is local to the function.
- Every task is checked.

ANSWER / ONE POSSIBLE SOLUTION

```
function countActive(tasks) {
  let count = 0;
  for (const task of tasks) {
    if (task.done === false) count++;
  }
  return count;
}
console.log(countActive(tasks)); // 1
```

Arrow functions are another notation

They are common in browser callbacks.

CONCEPT

Function declaration

function readTitle(task) { return task.title; }

Arrow form

const readTitle = task => task.title;

Return rule

A short expression body returns implicitly; a { } body needs return.

Arrow functions do not mean that code is automatically asynchronous.

Compare two function styles

Both examples return the task title.

READ THE CODE

- Both accept one parameter.
- The arrow form is concise.
- Both print Plan in this example.

BROWSER CONSOLE

```
function readTitle(task) {
  return task.title;
}
const shortRead = task => task.title;
console.log(readTitle(tasks[0]));
console.log(shortRead(tasks[0]));
```

A callback is a function passed as a value

Another operation decides when to call it.

CONCEPT

Passing

readTitle supplies the function without calling it.

Calling

readTitle(task) invokes the function now.

Applications

Array methods and event listeners use callbacks.

We use the same idea later in addEventListener.

map creates a new array

Turn tasks into a list of titles.

READ THE CODE

- map visits each task.
- The arrow returns task.title.
- The original tasks remain objects.

BROWSER CONSOLE

```
const titles = tasks.map(task => task.title);
console.log(titles);
// ["Plan", "Review"]
```

Practice 8 — Transform values

Use map with a clear returned result.

YOUR TURN

TASK

1. Start from the array tasks.
2. Create labels with each title followed by its status.
3. Store labels in a new array.
4. Print the result.

EXPECTED RESULT

["Plan: Active", "Review: Completed"]

HINT / CHECK

Use a template literal and statusLabel(task.done).

Solution — Map task summaries

Compare your code only after attempting the exercise.

READ THE CODE

- Braces mean we must write return.
- Without return, map collects undefined values.

ANSWER / ONE POSSIBLE SOLUTION

```
const labels = tasks.map(task => {
  return `${task.title}: ${statusLabel(task.done)}`;
});
console.log(labels);
```

Common beginner mistakes

Predict the behavior, then test it.

INCORRECT PATTERN	WHAT GOES WRONG
const n = 1; n = 2;	Reassignment error
tasks[2].title	tasks[2] may be undefined
tasks.map(t => { t.title; })	No return: undefined results
console.log("2" + 3)	Prints 23, not 5

The Console error usually identifies the source line. Read it before changing code.

Checkpoint 2 — Your data model

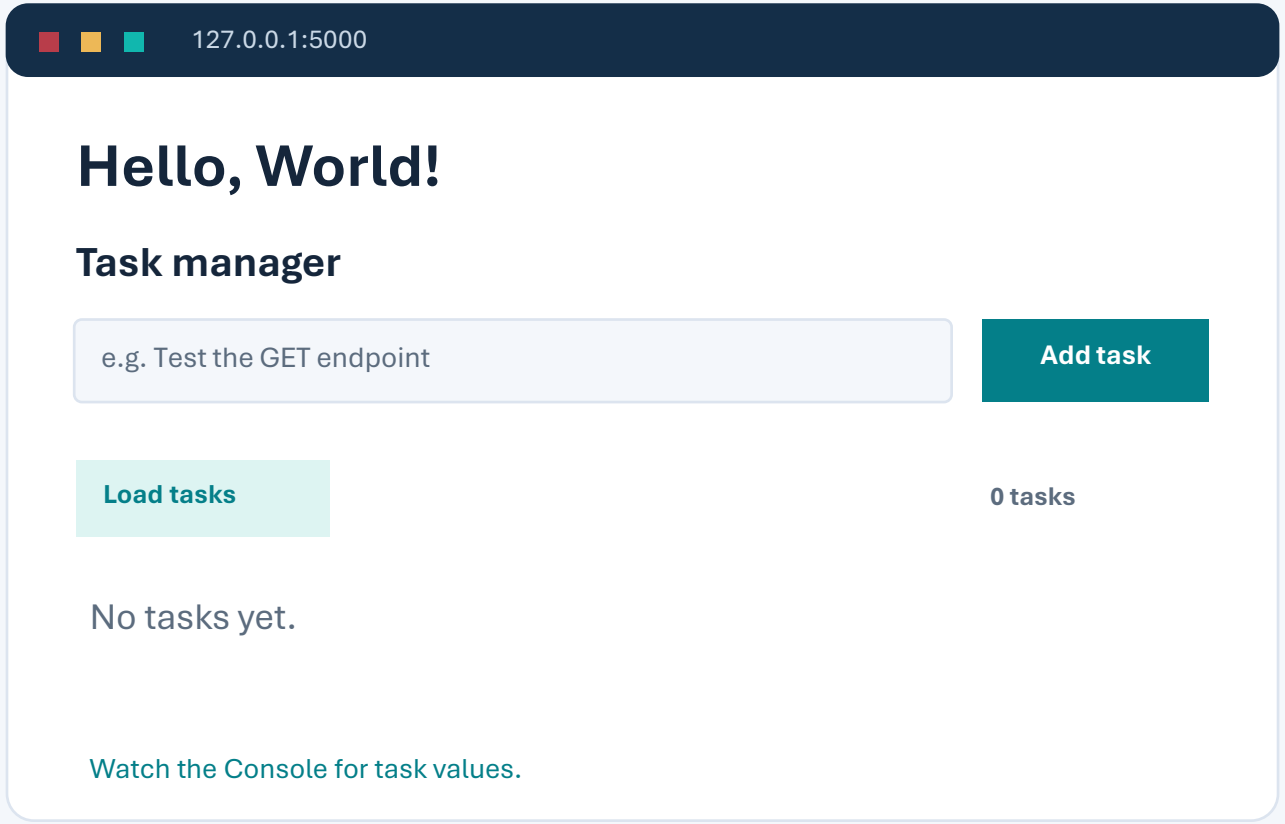
Run the variables, functions and array exercises.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/02_variables_and_functions.js

- Logs task names and statuses
- No click behavior yet
- Later the same objects will become DOM elements



04

DOM and events

Make the HTML page respond to the user.

Goal: connect real page elements to JavaScript.

What is the DOM?

The browser represents the HTML document as objects.

1

HTML

<h1>...</h1>

›

2

DOM

An element object

›

3

JavaScript

Find the element

›

4

UI

Change its text

The DOM is a browser API. JavaScript is the language used to interact with it.

Selectors find HTML elements

You already know CSS selectors; JavaScript can reuse them.

CONCEPT

#id selector

document.querySelector("#page-title")

Class selector

document.querySelector(".panel")

No match

querySelector returns null if nothing matches.

Always verify the id in HTML matches the selector string.

Read a heading from the page

An element reference lets us update it.

READ THE CODE

- querySelector finds the h1.
- textContent reads or writes text.
- Reload to reset the initial HTML.

APP.JS

```
const heading = document.querySelector("#page-title");
console.log(heading.textContent);
heading.textContent = "My JavaScript App";
```

Use textContent for user-provided text

Avoid treating task titles as markup.

SAFER DISPLAY	DANGEROUS CONFUSION
element.textContent = title;	element.innerHTML = title;
Displays literal text	Parses text as HTML markup
Good default for task names	Requires deliberate sanitization

The final project creates elements and assigns textContent for task titles.

Practice 9 — Change the heading

Your first visible JavaScript result.

YOUR TURN

TASK

1. Select #page-title.
2. Set its textContent to "JavaScript is running".
3. Reload the browser.
4. Explain why the original HTML has not changed on disk.

EXPECTED RESULT

The visible heading becomes JavaScript is running.

HINT / CHECK

The script modifies the browser DOM after the HTML loads.

Solution — Change a heading

Compare your code only after attempting the exercise.

READ THE CODE

- The saved HTML still says Hello, World!.
- JavaScript changes the in-memory DOM.

APP.JS

```
const heading = document.querySelector("#page-title");
heading.textContent = "JavaScript is running";
```

An event connects a user action to code

A browser can notify a function about an interaction.

CONCEPT

Event type

click, input and submit are common event names.

Listener

An event handler registered using addEventListener.

Callback

A function the browser calls when the event occurs.

Registration does not run the handler immediately.

The click-event timeline

Follow the exact order.

1

Select button

querySelector

2

Register

addEventListener

3

User clicks

Browser emits event

4

Handler runs

Callback executes

The callback runs because a browser event occurs—not because the script reaches the registration line.

Listen to a button click

Look at the handler function separately.

READ THE CODE

- announceClick is a function.
- Passing it does not call it.
- Click the button to run it.

APP.JS

```
const button = document.querySelector("#load-button");
function announceClick() {
  console.log("Button clicked!");
}
button.addEventListener("click", announceClick);
```

Registering vs calling a function

A pair of parentheses changes the meaning.

SYNTAX	MEANING
button.addEventListener("click", show);	Pass the function; browser calls later
show();	Run show immediately
() => show();	Pass a callback that calls show later

A beginner error is using show() instead of show in addEventListener.

Form inputs contain values

Read what the user typed, not the HTML placeholder.

CONCEPT

input.value

The current text in a form field.

trim()

Removes whitespace at the ends of a string.

Validation

Check for an empty title before creating a task.

" ".trim() becomes the empty string "".

Read the input value

A value changes when the user types.

READ THE CODE

- The selector matches the HTML id.
- value is current user input.
- trim() removes surrounding spaces.

APP.JS

```
const input = document.querySelector("#task-input");
const title = input.value.trim();
console.log(title);
```

Why form submission needs preventDefault

Browser forms have a default behavior.

CONCEPT

submit event

Fires when the form is submitted.

Default action

A normal submit may navigate or reload the page.

preventDefault()

Stops that action so our JavaScript can handle the form.

Use a submit listener on the form rather than only a click listener on the button.

A form event handler

Read title and display a message.

READ THE CODE

- event is provided by the browser.
- preventDefault stops page navigation.
- The handler reads the current value.

APP.JS

```
const form = document.querySelector("#task-form");
form.addEventListener("submit", event => {
  event.preventDefault();
  const title = input.value.trim();
  console.log("Submitted:", title);
});
```

The HTML ids used in this course

Keep these ids consistent in every example.

ELEMENT	SELECTOR
Page heading	#page-title
Task form	#task-form
Task title input	#task-input
Task list	#task-list
Task count	#task-count
Status message	#status
Load button	#load-button

All selectors match the companion project exactly.

Change the status paragraph on submit

Join DOM selection, form handling and text.

READ THE CODE

- Select the status paragraph.
- Read the input after submit.
- Set textContent to show feedback.

APP.JS

```
const status = document.querySelector("#status");
form.addEventListener("submit", event => {
  event.preventDefault();
  const title = input.value.trim();
  status.textContent = `You entered: ${title}`;
});
```

Practice 10 — First interactive form

Work only with existing HTML elements.

YOUR TURN

TASK

1. When Add task is submitted, prevent page reload.
2. Read and trim #task-input.
3. If blank, show "Please enter a title".
4. Otherwise show "You entered: ...".

EXPECTED RESULT

The status changes after each valid submit.

HINT / CHECK

Use if / else inside the submit handler.

Solution — Validate a form

Compare your code only after attempting the exercise.

READ THE CODE

- A nonempty string counts as true.
- For stricter style, use if (title === "").

APP.JS

```
form.addEventListener("submit", event => {
  event.preventDefault();
  const title = input.value.trim();
  status.textContent = title
    ? `You entered: ${title}`
    : "Please enter a title";
});
```

Checkpoint 3 — DOM + events

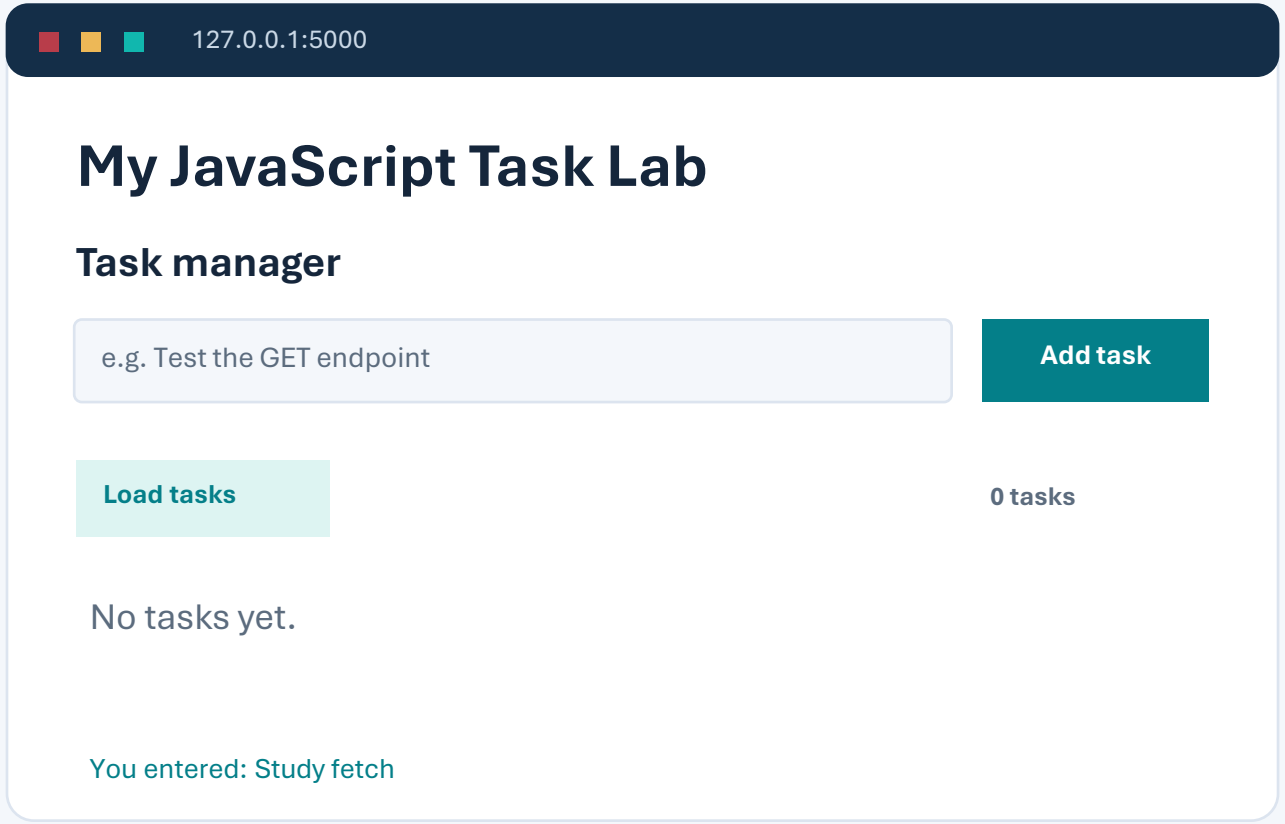
You can now explain a complete interaction.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/03_dom_events.js

- Heading changes on page load
- Form submission updates status
- Load button displays feedback



05

Build the task list locally

Before HTTP, learn how data appears on the page.

Goal: render task objects and add new ones in browser memory.

Create elements without writing HTML strings

The browser can create new DOM nodes.

CONCEPT

createElement

document.createElement("li") creates a list item.

textContent

Sets the visible title safely.

append

Attaches the new element into the task list.

The list must exist in the HTML, but its task rows can come entirely from JavaScript.

Create one task row

Start with one object, then display it.

READ THE CODE

- li is an element object.
- The title is plain text.
- append makes the row visible.

APP.JS

```
const list = document.querySelector("#task-list");
const task = { id: 1, title: "Plan", done: false };
const li = document.createElement("li");
li.textContent = task.title;
list.append(li);
```

A render function turns data into DOM

Separate the data from its representation.

CONCEPT

Data

The tasks array contains the current objects.

Clear

replaceChildren() removes old list rows.

Draw

Loop through tasks and append one li for each.

You can call renderTasks again whenever the array changes.

Render an array as list items

This is the core rendering pattern.

READ THE CODE

- Clear previous rows.
- Create a new li for each task.
- Append each row into the ul.

APP.JS

```
function renderTasks() {
  list.replaceChildren();
  for (const task of tasks) {
    const li = document.createElement("li");
    li.textContent = task.title;
    list.append(li);
  }
}
```

Add a task to an array

This version is local—there is no API yet.

READ THE CODE

- push adds an object.
- nextId gives each task an id.
- Render again to update the page.

APP.JS

```
let nextId = 3;
function addLocalTask(title) {
  tasks.push({ id: nextId, title, done: false });
  nextId++;
  renderTasks();
}
```

Practice 11 — Local task creation

Turn form text into a row without a server.

YOUR TURN

TASK

1. Start with an array of two task objects.
2. Render their titles in #task-list.
3. On submit, create a new object with done false.
4. Clear input, rerender and update task count.

EXPECTED RESULT

The new task appears immediately, but disappears after reload.

HINT / CHECK

Local variables reset when a page loads again.

Solution — Local form handler

Compare your code only after attempting the exercise.

READ THE CODE

- The addLocalTask function updates the array.
- Its renderTasks call refreshes the list.

APP.JS

```
form.addEventListener("submit", event => {
  event.preventDefault();
  const title = input.value.trim();
  if (!title) return;
  addLocalTask(title);
  input.value = "";
});
```

Checkpoint 4 — Local task manager

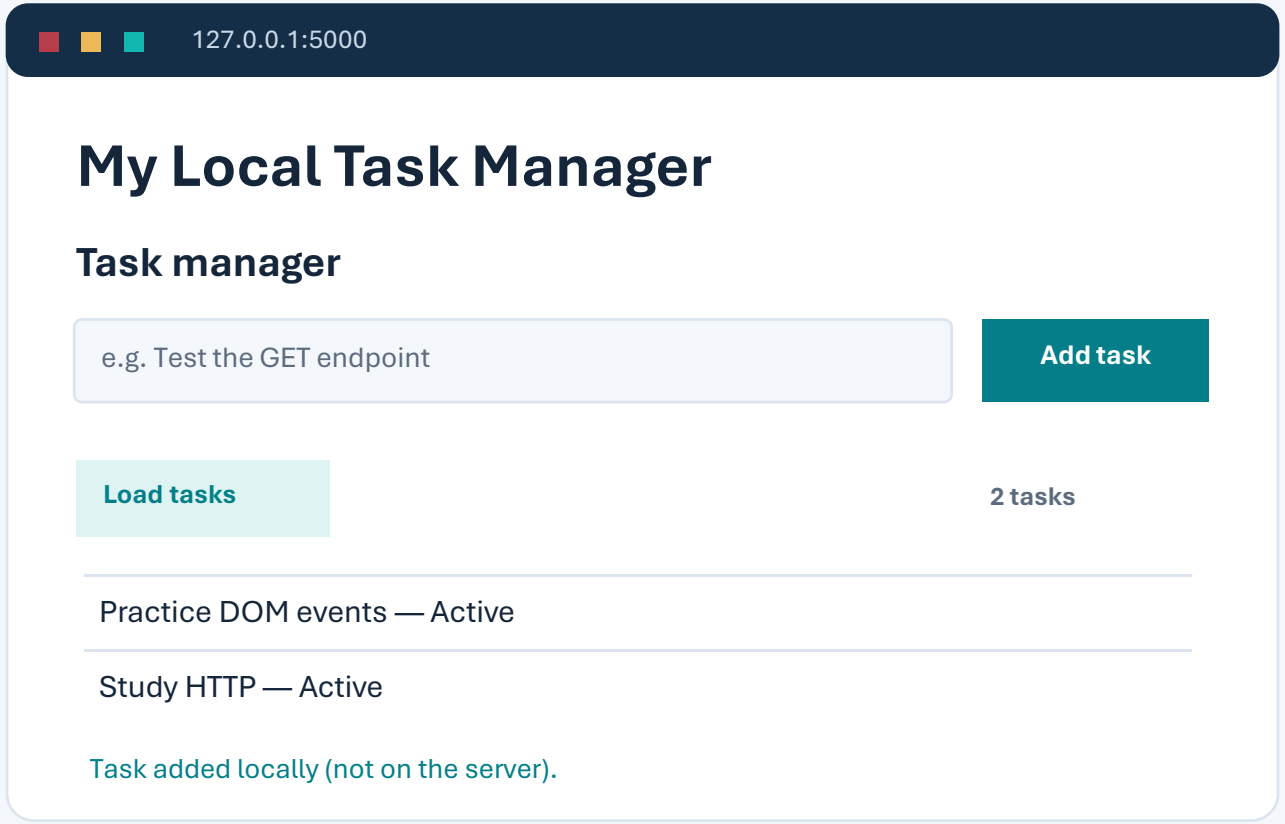
The app is interactive but nothing is stored on a server.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/04_local_tasks.js

- Click Add task and a row appears
- Reload and the new row disappears
- Next we will replace local data with API data



Why we need an API next

Browser memory is temporary; Flask and SQLite save server-side data.

1

User

Adds a task

2

Browser memory

tasks.push(...)

3

Reload

Memory is recreated

4

Flask + SQLite

Tasks persist after committed writes

Browser data disappear on reload; Flask persists committed data in its own database regardless of the frontend server.

06

JSON, HTTP and asynchronous code

Now connect the independent JavaScript site to the SAME Session 5 Flask Task API.

Goal: understand the request/response contract before writing fetch.

Reconnect two independent applications

Only HTTP + JSON connects the frontend to the Flask API and SQLite database.

Frontend :5500

index.html + app.js

fetch + JSON

Absolute URL, cross-origin

Backend :5000

Flask task_api routes

SQLAlchemy

store.py + models.py

SQLite

instance/task_manager.db

No shared directory, no frontend Python import, and no backend JavaScript import.

Two folders — no nested frontend

The existing Session 5 backend remains a separate project; frontend is a sibling.

```
IN432_Two_Independent_Applications/
  backend/
    task_api/
      __init__.py
      routes.py
      models.py
      store.py
    instance/task_manager.db
    requirements.txt
  frontend/
    index.html
    styles.css
    config.js
    app.js
```

Frontend has NO Flask imports or server-side Python dependencies.

http://127.0.0.1:5500/ → API: http://127.0.0.1:5000

Independent deployment. Flask never serves the HTML page in this version.

Run the applications independently

Two processes, two terminals, two ports; either one can stop while the other runs.

Terminal A — backend/

```
python -m flask --app task_api db-info
```

Start Flask — port 5000

```
python -m flask --app task_api run --debug --port 5000
```

Terminal B — frontend/

```
python -m http.server 5500 --bind 127.0.0.1
```

Open http://127.0.0.1:5500/; the page sends API requests to port 5000.

TWO INDEPENDENT APPS

Same computer ≠ same origin

The browser checks the scheme, hostname and port of the page and requested API.

FRONTEND

http://127.0.0.1:5500

Static HTML, CSS and JS

RUNS IN A DIFFERENT PROCESS

↔

BACKEND

http://127.0.0.1:5000

Flask, SQLAlchemy and SQLite

RUNS IN A DIFFERENT PROCESS

CONSEQUENCE: Cross-origin fetch requires a backend CORS response.

Use a full URL, not /api/tasks alone

A relative URL points back to the frontend server that supplied the page.

WRONG TARGET

```
fetch("/api/tasks")
```

Requests port 5500 (the HTML server). It does not implement /api/tasks.

CORRECT TARGET

```
// frontend/config.js
const API_BASE_URL =
  "http://127.0.0.1:5000";
```

fetch(`${API_BASE_URL}/api/tasks`) requests the Flask server.

KEY POINT: The frontend knows only the API origin and its HTTP/JSON contract.

Trace one cross-origin POST to SQLite

Frontend host and backend host/port need not be the same.

1. Browser	fetch(`${API_BASE_URL}/api/tasks`, {...})
2. Flask route	Read JSON + validate title
3. ORM store	Task(title=...) → db.session.add(task)
4. Transaction	db.session.commit() saves the row
5. Response	201 Created + Location + task JSON

The API validates and commits independently of where its caller is hosted.

The API contract remains unchanged

Only the BASE URL changes; Flask still defines GET, POST, PATCH, DELETE and optional PUT /project.

CLIENT ACTION	METHOD / ROUTE	EXPECTED
Load tasks	`GET /api/tasks`	200 JSON array
Create task	`POST /api/tasks`	201 + Location, committed
Complete / rename	`PATCH /api/tasks/<id>`	200 changed task
Delete task	`DELETE /api/tasks/<id>`	204 empty body
Optional: assign project	`PUT /api/tasks/<id>/project`	Part B, schema version 2

PUT /api/tasks/<id> is NOT a task-update route in the Session 5 project.

Test independence AND persistence

The page and the Flask process can restart independently.

01 POST on port 5500	Create a disposable task; record its actual id.
02 Restart frontend only	Reload port 5500; it reads from the same Flask API.
03 Stop Flask only	The page still loads; API requests fail while Flask is offline.
04 Restart Flask	GET returns the same committed task from SQLite.
05 PATCH and DELETE	Update an existing task; delete only a disposable record.

Show two ports in DevTools and inspect the database path with db-info.

Where an API fits

The frontend runs on :5500; a full URL targets Flask on :5000.

1

Browser

Runs app.js

›

2

fetch

Creates HTTP request

›

3

Flask task_api

Validates and uses ORM

›

4

SQLite

Persists committed changes

The page and API are distinct origins; JSON over HTTP is their only interface.

Client vs server responsibilities

JavaScript owns the UI; Flask owns the REST API and persistence, wherever each is deployed.

CLIENT — BROWSER	SERVER — API
Read user input	Validate JSON request
Draw task rows	Validate and store task
Send HTTP requests	Choose status code
Display success or errors	Commit to SQLite; return JSON

Use the completed Session 5 backend. Do not create a second server for this lab.

HTTP request and response

GET is handled by the Flask backend, not the frontend static server.

CONCEPT

Method and URL

GET /api/tasks identifies the operation and resource.

Headers

Content-Type indicates the format of the request body.

Body and status

JSON may be sent in the body; server returns status + data.

An HTTP 404 is not a JavaScript syntax error: it is a server response.

CRUD methods on our Flask API

The method/endpoint is defined by Flask, independently of the frontend implementation.

CRUD	HTTP METHOD	OUR TASK ACTION
Read	GET	Load one or all tasks
Create	POST	Add a new task
Update task	PATCH	Rename title or change done
Assign project (Part B)	PUT	PUT /api/tasks/:id/project
Delete	DELETE	Remove a task

Core task CRUD: GET, POST, PATCH, DELETE. Project assignment uses PUT only after schema v2.

Actual task_api endpoints

The URLs below are API paths; prefix them with API_BASE_URL in frontend code.

METHOD	URL	SUCCESS
GET	/api/tasks	200 → array of tasks
GET	/api/tasks/1	200 → one task
POST	/api/tasks	201 → created task
PATCH	/api/tasks/1	200 → updated task title
PUT*	/api/tasks/1/project	200 → assignment (*Part B)
DELETE	/api/tasks/1	204 → no response body

Examples use illustrative IDs; core CRUD works in Part A, PUT /project needs Part B.

JSON is a data-exchange format

It resembles a JavaScript object, but it is text.

CONCEPT

JSON object

{"title":"Learn fetch","done":false}

JSON array

[{"id":1,"title":"Plan","done":false}]

Syntax rules

Property names and strings use double quotes. No functions or comments.

JSON is data—not executable JavaScript code.

JavaScript object vs JSON text

This difference matters when sending requests.

READ THE CODE

- The JS object is in memory.
- stringify converts data to JSON text.
- HTTP requests send encoded data.

BROWSER CONSOLE

```
const task = { title: "Learn fetch", done: false };
const jsonText = JSON.stringify(task);
console.log(typeof task);     // object
console.log(typeof jsonText); // string
```

Convert JSON text into an object

JSON.parse works with a valid JSON string.

READ THE CODE

- data is a string.
- JSON.parse returns a JS value.
- Invalid JSON throws an error.

BROWSER CONSOLE

```
const data = '{"id":1,"title":"Plan","done":false}';
const task = JSON.parse(data);
console.log(task.title); // Plan
```

Important HTTP status codes

Browser JSON requests can cross origins when the API permits the frontend origin.

STATUS	MEANING	EXAMPLE HERE
200 OK	Successful response	GET / PATCH / PUT*
201 Created	New resource created	POST /api/tasks
204 No Content	Success, empty body	DELETE /api/tasks/:id
400 Bad Request	Invalid input or JSON	Blank title / malformed JSON
404 Not Found	Unknown endpoint or task	GET /api/tasks/999
500 Server Error	Internal server failure	Investigate server logs

Also inspect 415 for wrong content type; 501 can indicate unfinished Session 5 TODOs.

Network operations take time

The browser should not freeze while waiting.

CONCEPT

Synchronous code

An ordinary statement executes in sequence.

Promise

Represents an eventual result of an asynchronous operation.

await

Lets an async function wait for a Promise result.

fetch returns a Promise; it does not immediately return the task array.

The fetch lifecycle

Fetch waits for Flask's response over HTTP, even if the two servers are on different machines.

1

Start

fetch(url)

2

Wait

await response

3

Validate

response.ok

4

Decode

response.json()

5

Display

Render tasks

A response with HTTP 404 still produces a Response object; always inspect response.ok.

async and await: first example

Think of a function that awaits a network result.

READ THE CODE

- async permits await.
- fetch waits for Response.
- response.json() also returns a Promise.

APP.JS

```
async function loadTasks() {
  const response = await
fetch(`${API_BASE_URL}/api/tasks`);
  const tasks = await response.json();
  console.log(tasks);
}
loadTasks();
```

Use try/catch for failures

Network errors and JSON errors need user feedback.

CONCEPT

try

Contains work that might fail.

throw

Creates an explicit error condition.

catch

Handles a rejected Promise or thrown error.

fetch does not reject merely because the server replies 404.

A safer GET request

Check the status of the response from the independent Flask service.

READ THE CODE

- response.ok is true for 2xx.
- Throw if HTTP status is an error.
- catch shows what failed.

APP.JS

```
async function loadTasks() {
  try {
    const response = await fetch(`${API_BASE_URL}/api/tasks`);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const tasks = await response.json();
    console.log(tasks);
  } catch (error) {
    console.error(error.message);
  }
}
```

GET — read the task collection

No request body is required.

GET **/api/tasks**

REQUEST

GET /api/tasks
(no request body)

RESPONSE

200 OK
[]

(A new Session 5 database may have no tasks yet.)

Create a task with POST to populate the persistent database; GET returns saved rows.

Render tasks received as JSON

Use the function you already learned.

READ THE CODE

- The API supplies task objects.
- Assign results to the task array.
- Render the updated array.

APP.JS

```
async function loadTasks() {
  const response = await
fetch(`${API_BASE_URL}/api/tasks`);
  if (!response.ok) throw new Error("Load failed");
  tasks = await response.json();
  renderTasks();
}
```

Practice 12 — GET + render

Use real server data instead of local sample data.

YOUR TURN

TASK

1. Run python -m flask --app task_api run --debug and open the app URL.
2. Call fetch(`${API_BASE_URL}/api/tasks`) inside an async function.
3. Check response.ok and await response.json().
4. Render the titles in #task-list.

EXPECTED RESULT

The response may contain saved tasks or an empty array.

HINT / CHECK

Use checkpoint 05 if you need to recover your working files.

Checkpoint 5 — GET from API

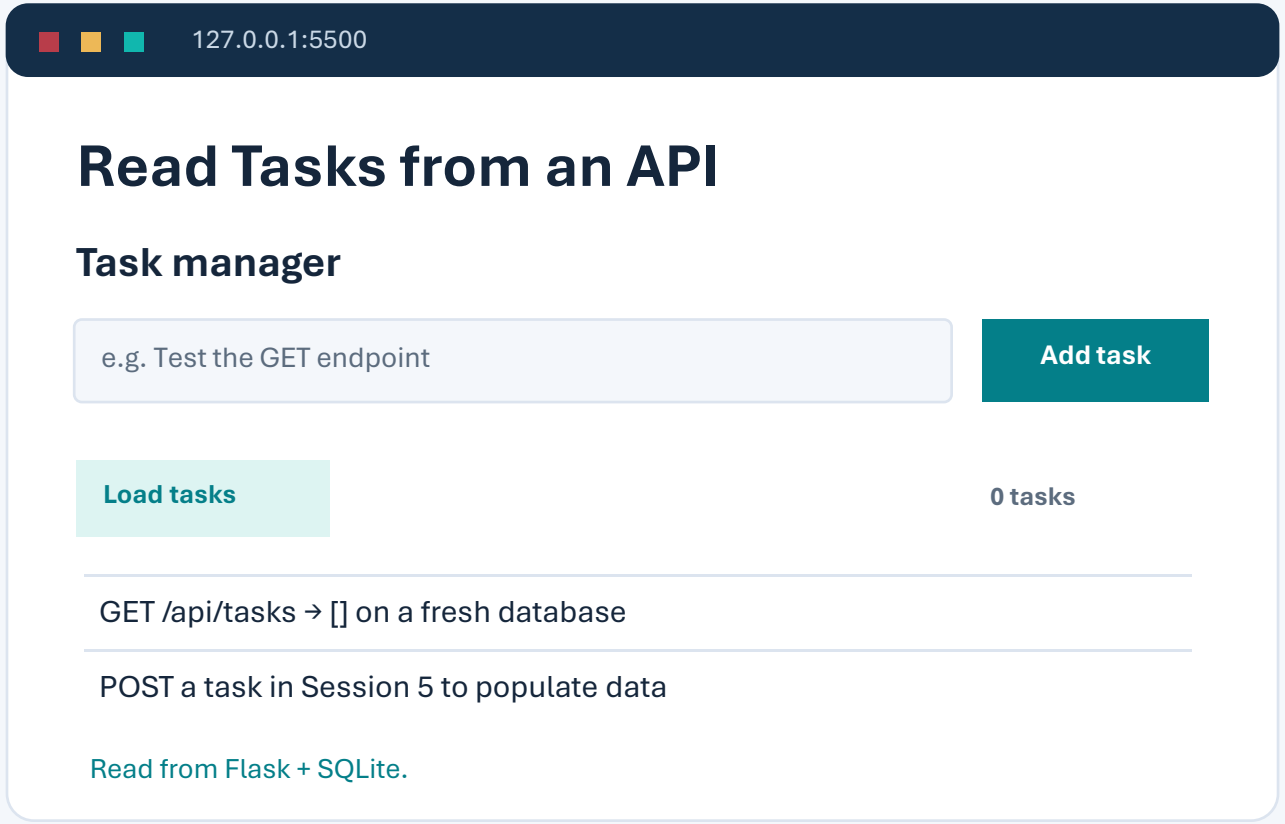
The page on port 5500 displays JSON returned by Flask on port 5000.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/05_get_tasks.js

- Click Load tasks to repeat GET
- Inspect Flask response in DevTools Network
- An empty list is normal before your first POST



Network panel: what to inspect

Network tools show the backend request URL: http://127.0.0.1:5000/api/tasks.

DEVTOOLS

Name + method

Find /api/tasks and identify GET, POST, etc.

Status

Check 200, 201, 204 or an error status.

Headers + payload

Inspect Content-Type and JSON sent/received.

Select Fetch/XHR in the Network panel to focus on API calls.

CORS and the JSON preflight

For cross-origin JSON POST/PATCH, the browser may send OPTIONS before the real request.

1

Browser

OPTIONS /api/tasks

Origin: http://127.0.0.1:5500

2

Flask

200 + CORS headers

Allow origin, methods and Content-Type

3

Browser

POST JSON /api/tasks

Server validates and commits task

4

Flask

201 Created + JSON

Browser checks and renders result

IMPORTANT: CORS is a browser sharing rule; it is not a substitute for authentication.

Where to configure CORS in Flask

Add a narrow allow-list to the existing Session 5 app factory; do not move the frontend.

backend/task_api/__init__.py • inside create_app()

```
allowed = {"http://127.0.0.1:5500"}

@app.after_request
def allow_frontend(response):
    origin = request.headers.get("Origin")
    if request.path.startswith("/api/") and origin in allowed:
        response.headers["Access-Control-Allow-Origin"] = origin
        response.headers["Access-Control-Allow-Headers"] = "Content-Type"
        response.headers["Access-Control-Allow-Methods"] = "GET, POST,
PATCH, PUT, DELETE, OPTIONS"
    return response
```

READ THE CODE

Allow only known frontend origins.

Flask answers OPTIONS for these routes.

CORS is not authentication.

See backend/CORS_INTEGRATION.md .

SAFETY: Never expose the classroom Flask debug server to the public Internet.

07

Complete CRUD with fetch

Create, read, update and delete persistent tasks using the Flask API on another origin.

Goal: every button changes server data through HTTP.

One complete action across the app

One task travels from a standalone JavaScript form to Flask and SQLite.

1

Form

Read title

2

POST

Send JSON

3

Flask ORM

Commit task

4

GET

Reload tasks

5

DOM

Redraw rows

The UI server and API server never need to share a filesystem.

POST — create a new task

POST sends JSON from :5500 to :5000, then Flask commits it to SQLite.

POST **/api/tasks**

REQUEST

Content-Type: application/json

```
{ "title": "Study fetch" }
```

RESPONSE

201 Created

```
{ "id": 3,
  "title": "Study fetch",
  "done": false }
```

Flask validates, commits and returns 201 + a Location header after success.

POST needs three key options

fetch takes the URL plus an options object.

CONCEPT

method

"POST" tells the API to create a resource.

headers

"Content-Type": "application/json" describes the request body.

body

JSON.stringify({ title }) turns an object into JSON text.

Do not send the JavaScript object directly as the request body.

Write a POST request

The browser sends POST to the full Flask API URL, not the frontend server.

READ THE CODE

- POST targets the collection.
- JSON.stringify creates the body.
- 201 response contains new task.

INSIDE AN ASYNC FUNCTION

```
const response = await
fetch(`${API_BASE_URL}/api/tasks`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Study fetch" })
});
if (!response.ok) throw new Error("Create failed");
const created = await response.json();
```

Where POST belongs in your interface

The standalone form calls an async handler that requests the external API.

CONCEPT

Collect input

const title = input.value.trim()

Validate

If blank, show an error without sending POST.

Update UI

After success, clear input and call loadTasks().

Handle the form submit event and prevent the browser’s default submission.

Connect the form to POST

The event handler can use await because it is async.

READ THE CODE

- Prevent page reload.
- Read, validate, send JSON.
- Reload tasks after success.

APP.JS

```
form.addEventListener("submit", async event => {
  event.preventDefault();
  const title = input.value.trim();
  if (!title) return;
  await fetch(`${API_BASE_URL}/api/tasks`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title })
  });
  input.value = "";
  await loadTasks();
});
```

Practice 13 — GET and POST

Connect the independently served form to the same Session 5 Flask API.

YOUR TURN

TASK

1. Start from checkpoint 05.
2. Add a submit listener to #task-form.
3. POST JSON {title} to /api/tasks.
4. Check response.ok, clear input, reload tasks.

EXPECTED RESULT

After POST, the new task survives browser reload and Flask restart.

HINT / CHECK

Try checkpoint 06 after attempting the code.

Checkpoint 6 — GET + POST

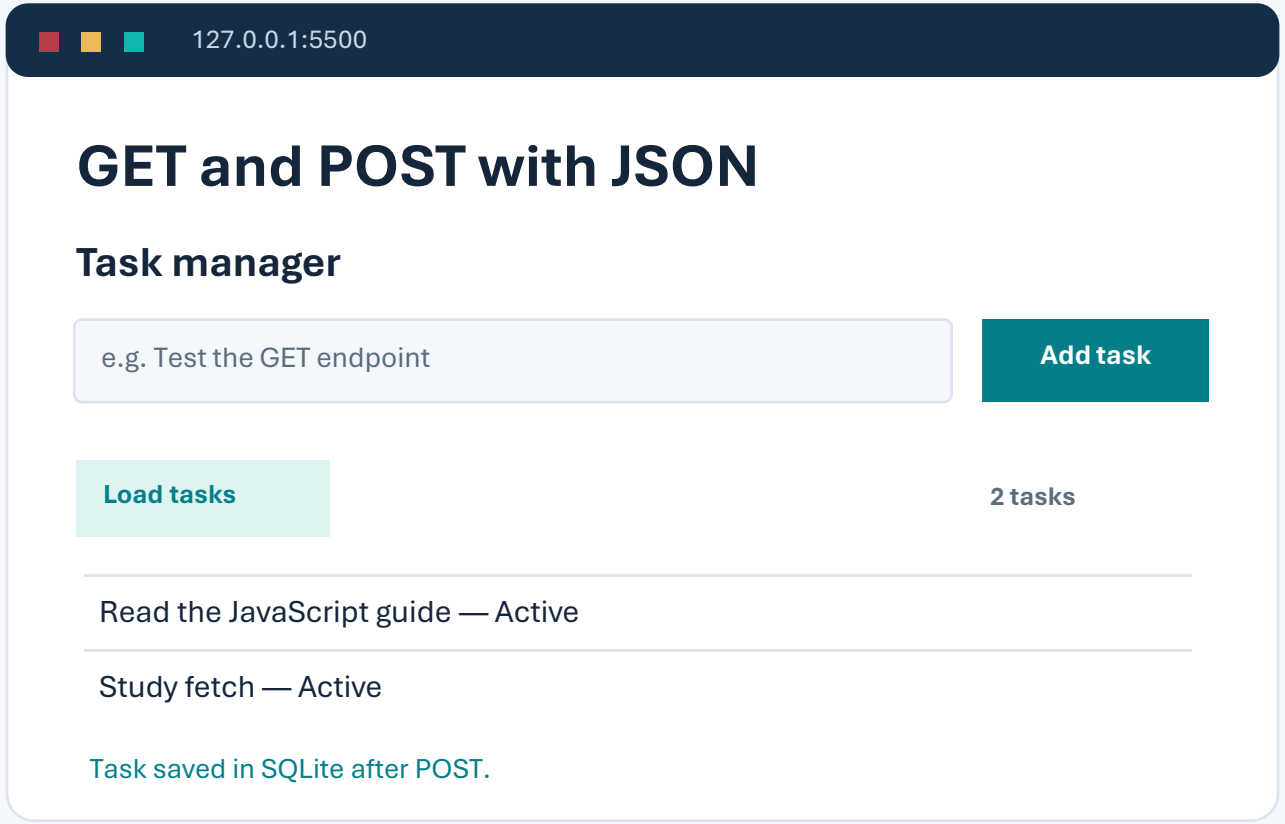
Task creation persists in SQLite, regardless of whether the frontend reloads.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/06_get_post_tasks.js

- Network shows POST 201
- A subsequent GET returns the new row
- The created row survives a Flask restart



Why PATCH for tasks and PUT for projects?

The real backend defines which method each endpoint supports.

CONCEPT

PATCH /api/tasks/:id

Partial changes to title and/or done (Part A).

PUT /api/tasks/:id/project

Assign/unassign an existing Project (Part B).

Important distinction

PUT on /api/tasks/:id is NOT a Session 5 core route.

Do not assume PUT and PATCH are interchangeable. Always follow the server contract.

PUT — assign a task to a project

OPTIONAL: requires Session 5 Part B (Project model + schema version 2).

PUT **/api/tasks/1/project**

REQUEST

Content-Type: application/json

```
{ "project_id": 7 }
```

RESPONSE

200 OK

```
{ "id": 1, "title": "Study fetch",
  "done": false, "project_id": 7 }
```

Use actual task and project IDs; 1 and 7 are examples, not fixed records.

Send the optional PUT relationship request

Update an assignment, not the task title or done flag.

READ THE CODE

- API_BASE_URL comes from frontend/config.js.
- The request sends project_id, not title.
- Only after schema version 2 and existing Project id.

INSIDE AN ASYNC FUNCTION

```
const projectId = 7; // replace with an existing id
const response = await
fetch(`${API_BASE_URL}/api/tasks/${task.id}/project`,
{
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ project_id: projectId })
});
if (!response.ok) throw new Error(`HTTP
${response.status}`);
```

PATCH — update only one field

Part A PATCH updates a task while preserving other fields.

PATCH **/api/tasks/1**

REQUEST

Content-Type: application/json

```
{ "done": true }
```

RESPONSE

200 OK

```
{ "id": 1,
  "title": "Read the JavaScript guide",
  "done": true }
```

The existing title is preserved on the server.

Send a PATCH status change

Flask validates the Boolean and commits it to SQLite.

READ THE CODE

- ! reverses true/false.
- PATCH sends only done.
- Use response.ok before treating it as success.

INSIDE AN ASYNC FUNCTION

```
const nextDone = !task.done;
await fetch(`${API_BASE_URL}/api/tasks/${task.id}`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ done: nextDone })
});
```

Where a completion button gets its task

Create a button while rendering each row.

1

Loop

Read task

2

Button

Create element

3

Listener

Capture task.id

4

PATCH

Send done flag

5

Refresh

loadTasks()

An arrow function such as () => toggleTask(task) calls the handler only after clicking.

Connect a button to a task

The callback uses the current loop value.

READ THE CODE

- Make one button per row.
- The callback remembers its task.
- toggleTask sends PATCH.

INSIDE RENDERTASKS()

```
const toggle = document.createElement("button");
toggle.textContent = task.done ? "Reopen" : "Complete";
toggle.addEventListener("click", () => {
  toggleTask(task);
});
li.append(toggle);
```

Practice 14 — Complete a task

Each task must get its own button.

YOUR TURN

TASK

1. Create a Complete / Reopen button during rendering.
2. On click, PATCH {done: !task.done}.
3. Check the HTTP status.
4. Reload tasks and show updated status.

EXPECTED RESULT

Completed tasks change label and styling.

HINT / CHECK

Start with toggleTask(task) as an async function.

DELETE — remove a task

Flask deletes the chosen task row and commits the change.

DELETE **/api/tasks/1**

REQUEST

DELETE /api/tasks/1
(no request body)

RESPONSE

204 No Content

(no response JSON)

Do not try to parse an empty 204 response body.

Send DELETE and refresh

Send DELETE to Flask on port 5000, then reload tasks.

READ THE CODE

- DELETE specifies the task in the URL.
- The server returns 204.
- Refresh after success.

INSIDE AN ASYNC FUNCTION

```
const response = await
fetch(`${API_BASE_URL}/api/tasks/${task.id}`, {
  method: "DELETE"
});
if (!response.ok) throw new Error("Delete failed");
await loadTasks();
```

Why a 204 response needs special handling

Successful HTTP responses do not always contain JSON.

CONCEPT

GET 200

Can decode response.json().

POST 201

Can decode the created task JSON.

DELETE 204

Body is empty; skip response.json().

A helper can return null when response.status === 204.

One helper for HTTP + JSON

The helper builds a full API URL from the backend origin.

READ THE CODE

- path is an API route, such as /api/tasks.
- API_BASE_URL targets the Flask server.
- 204 has no JSON response body.

APP.JS

```
async function requestJSON(path, options = {}) {
  const response = await
fetch(`${API_BASE_URL}${path}`, options);
  if (!response.ok) throw new Error(`HTTP
${response.status}`);
  if (response.status === 204) return null;
  return response.json();
}
```

Build JSON request options once

Avoid repeating headers and stringify.

READ THE CODE

- The helper takes method and value.
- It returns an options object.
- Use for POST and PATCH; optional PUT /project later.

APP.JS

```
function jsonOptions(method, value) {
  return {
    method,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(value)
  };
}
await requestJSON("/api/tasks", jsonOptions("POST",
{title}));
```

Errors should appear in the page

Students should not have to watch only the Console.

CONCEPT

Loading

Inform the user while a request is pending.

Success

Display what action succeeded.

Error

Read error.message and show helpful feedback.

Leave the typed title in the form when POST fails so the user can retry.

Catch and display an API error

Show a useful message instead of failing silently.

READ THE CODE

- catch handles thrown/rejected errors.
- Update the status paragraph.
- Only clear the input after success.

INSIDE SUBMIT HANDLER

```
try {
  await requestJSON("/api/tasks", jsonOptions("POST",
{title}));
  input.value = "";
  await loadTasks();
} catch (error) {
  status.textContent = `Could not add:
${error.message}`;
}
```

An accessible and safer task row

Create elements rather than putting user input into HTML strings.

CONCEPT

Use textContent

The task title remains text, not HTML.

Use real buttons

Buttons are keyboard-accessible by default.

Use clear labels

Complete, Reopen, Rename and Delete describe actions.

The supplied final app adds aria-label attributes to per-task buttons.

Final rendering pattern

The source reference contains the complete function.

READ THE CODE

- Create a fresh li each time.
- Set title as text.
- Then attach action buttons to li.

APP.JS

```
function renderTasks() {
  list.replaceChildren();
  for (const task of tasks) {
    const li = document.createElement("li");
    const title = document.createElement("span");
    title.textContent = task.title;
    li.append(title);
    list.append(li);
  }
}
```

The complete application data flow

The UI server is independent; SQLite writes belong to Flask.

1

Event

Click or submit

›

2

Handler

Validate input

›

3

Flask + ORM

Validate + commit

›

4

Response

Check status

›

5

Render

GET + redraw

frontend/solution/app.js calls API_BASE_URL; backend/task_api implements the original Session 5 API.

Checkpoint 7 — Full CRUD Task Manager

Demonstrate GET, POST, PATCH and DELETE with durable data.

CHECKPOINT

SOURCE TO COMPARE

checkpoints/07_full_crud.js

- GET loads tasks and POST creates one
- PATCH toggles and renames task fields
- DELETE removes; restart Flask to verify persistence

